

Oort: Efficient Federated Learning via Guided Participant Selection

Fan Lai, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury,
University of Michigan

https://www.usenix.org/conference/osdi21/presentation/lai

This paper is included in the Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation.

July 14–16, 2021

978-1-939133-22-9

Open access to the Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation is sponsored by USENIX.



Oort: Efficient Federated Learning via Guided Participant Selection

Fan Lai, Xiangfeng Zhu, Harsha V. Madhyastha, Mosharaf Chowdhury
University of Michigan

Abstract

Federated Learning (FL) is an emerging direction in distributed machine learning (ML) that enables in-situ model training and testing on edge data. Despite having the same end goals as traditional ML, FL executions differ significantly in scale, spanning thousands to millions of participating devices. As a result, data characteristics and device capabilities vary widely across clients. Yet, existing efforts randomly select FL participants, which leads to poor model and system efficiency.

In this paper, we propose Oort to improve the performance of federated training and testing with guided participant selection. With an aim to improve time-to-accuracy performance in model training, Oort prioritizes the use of those clients who have both data that offers the greatest utility in improving model accuracy and the capability to run training quickly. To enable FL developers to interpret their results in model testing, Oort enforces their requirements on the distribution of participant data while improving the duration of federated testing by cherry-picking clients. Our evaluation shows that, compared to existing participant selection mechanisms, Oort improves time-to-accuracy performance by 1.2×-14.1× and final model accuracy by 1.3%-9.8%, while efficiently enforcing developer-specified model testing criteria at the scale of millions of clients.

1 Introduction

Machine learning (ML) today is experiencing a paradigm shift from cloud datacenters toward the edge [18, 40]. Edge devices, ranging from smartphones and laptops to enterprise surveillance cameras and edge clusters, routinely store application data and provide the foundation for machine learning beyond datacenters. With the goal of not exposing raw data, large companies such as Google and Apple deploy *federated learning (FL)* for computer vision (CV) and natural language processing (NLP) tasks across user devices [2, 24, 30, 77]; NVIDIA applies FL to create medical imaging AI [49]; smart cities perform in-situ image training and testing on AI cameras to avoid expensive data migration [32, 38, 51]; and video streaming and networking communities use FL to interpret and react to network conditions [10, 76].

Although the life cycle of an FL model is similar to that in traditional ML, the underlying execution in FL is spread across thousands to millions of devices in the wild. Similar to traditional ML, the FL developer often first prototypes model architectures and hyperparameters with a proxy dataset. After selecting a suitable configuration, she can use federated training to improve model performance by training across a crowd of participants [18, 40]. The wall clock time for training a model to reach an accuracy target (i.e., time-to-accuracy) is still a key performance objective even though it may take significantly longer than centralized training [40]. To circumvent biased or stale proxy data in hyperparameter tuning [57], to inspect these models being trained, or to validate deployed models after training [75,76], developers may want to perform federated testing on the real-life client data, wherein enforcing their requirements on the testing set (e.g., N samples for each category or following the representative categorical distribution[1]) is crucial for them to reason about model performance under different data characteristics [20, 57].

Unfortunately, clients may not all be simultaneously available for FL training or testing [40]; they may have heterogeneous data distributions and system capabilities [18, 34]; and including too many may lead to wasted work and suboptimal performance [18] (§2). Consequently, a fundamental problem in practical FL is the *selection of a "good" subset of clients as participants*, where each participant locally processes its own data, and only their results are collected and aggregated at a (logically) centralized coordinator.

Existing works optimize for *statistical model efficiency* (i.e., better training accuracy with fewer training rounds) [22, 47, 59, 72] or *system efficiency* (i.e., shorter rounds) [54, 68], while randomly selecting participants. Although random participant selection is easy to deploy, unfortunately, it results in poor performance of federated training because of large heterogeneity in device speed and/or data characteristics. Worse, random participant selection can lead to biased testing sets and loss of confidence in results. As a result, developers often resort to more participants than perhaps needed [57, 73].

We present Oort for FL developers to enable guided participant selection throughout the life cycle of an FL model (§3). Specifically, Oort cherry-picks participants to improve time-to-accuracy performance for federated training, and it enables

[1] A categorical distribution is a discrete probability distribution showing how a random variable can take the result from one of K possible categories.

developers to specify testing criteria for federated model testing. It makes informed participant selection by relying on the information already available in existing FL solutions [40] with little modification.

Selecting participants for federated training is challenging because of the trade-off between heterogeneous system and statistical model utilities both across clients and of any specific client over time (as the trained model changes). First, simply picking clients with high statistical utility can lead to longer training rounds due to the coupled nature of client data and system performance. The challenge is further exacerbated by the large population, as capturing the latest utility of all clients is impractical. As such, we identify clients with high statistical utility, which is measured in terms of their most recent aggregate training loss, adjusted for spatiotemporal variations, and penalize the utility of a client if her system speed is likely to elongate the duration necessary to complete global aggregation. To navigate the sweet point of jointly maximizing statistical and system efficiency, we adaptively allow for longer training rounds to admit clients with higher statistical utility. We then employ an online exploration-exploitation strategy to probabilistically select participants among high-utility clients for robustness to outliers. Our design can accommodate diverse selection criteria (e.g., fairness), and deliver improvements while respecting privacy (§4).

Although FL developers often have well-defined requirements on their testing data, satisfying these requirements is not straightforward. Similar to traditional ML, developers may request a testing dataset that follows the global distribution to avoid testing on all clients [35, 57]. However, clients' data characteristics in some private FL scenarios may not be available [27, 77]. To preserve the deviation target of participant data from the global, Oort performs participant selection by bounding the number of participants needed. Second, for cases where clients' data characteristics are provided [51], developers can specify specific distribution of the testing set to debug model efficiency (e.g., using balanced distribution) [15, 78]. At scale, satisfying this requirement in FL suffers large overhead. Therefore, we propose a scalable heuristic to efficiently enforce developer requirements, while optimizing the duration of testing (§5).

We have integrated Oort with PySyft (§6) and evaluated it across various FL tasks with real-world workloads (§7). [2] Compared to the state-of-the-art selection techniques used in today's FL deployments [21, 73, 77], Oort improves time-to-accuracy performance by 1.2×-14.1× and final model accuracy by 1.3%-9.8% for federated model training, while achieving close to upper-bound statistical performance. For federated model testing, Oort can efficiently respond to developer-specified data distribution across millions of clients, and improves the end-to-end testing duration by 4.7× on average over state-of-the-art solutions.

[2] Oort is available at https://github.com/SymbioticLab/Oort.

Overall, we make the following contributions in this paper:

1. We highlight the tension between statistical and systems efficiency when selecting FL participants and present Oort to effectively navigate the tradeoff.
2. We propose participant selection algorithms to improve the time-to-accuracy performance of training and to scalably enforce developers' FL testing criteria.
3. We implement and evaluate these algorithms at scale in Oort, showing both statistical and systems performance improvements over the state-of-the-art.

2 Background and Motivation

We start with a quick primer on federated learning (§2.1), followed by the challenges it faces based on our analysis of real-world datasets (§2.2). Next, we highlight the key shortcomings of the state-of-the-art that motivate our work (§2.3).

2.1 Federated Learning

Training and testing play crucial roles in the life cycle of an FL model, whereas they have different criteria.

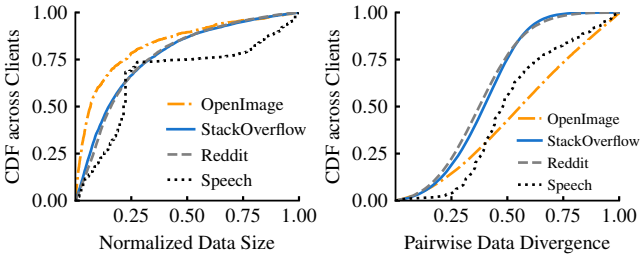
Federated model training aims to learn an accurate model across thousands to potentially millions of clients. Because of the large population size and diversity of user data and their devices in FL, training runs on a subset of clients (hundreds of participants) in each round, and often takes hundreds of rounds (each round lasts a few minutes) and several days to complete. For example, in Gboard keyboard, Google runs federated training of NLP models over weeks across 1.5 million end devices [4, 77]. For a given model, achieving a target model accuracy with less wall clock time (i.e., time-to-accuracy) is still the primary target [47, 63].

To inspect a model's accuracy during training (e.g., to detect cut-off accuracy), to validate the trained model before deployment [21, 73, 77], or to circumvent biased proxy data in hyperparameter tuning [15, 62], FL developers sometimes test model's performance on real-life datasets. Similar to traditional ML, developers often request the representativeness of the testing set with requirements like *"50k representative samples"* [15], or *"x samples of class y"* to investigate model performance on specific categories [78]. When the data characteristics of participants are not available, coarse-grained yet non-trivial requests, such as *"a subset with less than X% data deviation from the global"* are still informative [53, 57].

2.2 Challenges in Federated Learning

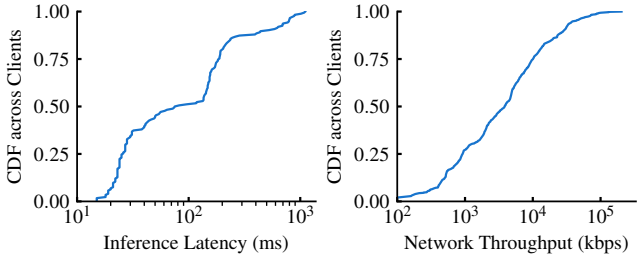
Apart from the challenges faced in traditional ML, FL introduces new challenges in terms of data, systems, and privacy.

Heterogeneous statistical data. Data in each FL participant is typically generated in a distributed manner under different contexts and stored independently. For example, images collected by cameras will reflect the demographics of each camera's location. This breaks down the widely-accepted assumption in traditional ML that samples are independent and identically distributed (i.i.d.) from a data distribution.



(a) Unbalanced data size. (b) Heterogeneous data distribution.

Figure 1: ***Client data differs in size and distribution greatly.***



(a) Heterogeneous compute capacity. (b) Heterogeneous network capacity.

Figure 2: ***Client system performance differs significantly.***

We analyze four real-world datasets for CV (OpenImage [3]) and NLP (StackOverflow [9], Reddit [8] and Google Speech [74]) tasks. Each consists of thousands or up to millions of clients and millions of data points. In each individual dataset, we see a high statistical deviation across clients not only in the quantity of samples (Figure 1(a)) but also in the data distribution (Figure 1(b)).[3]

Heterogeneous system performance. As individual data samples are tightly coupled with the participant device, in-situ computation on this data experiences significant heterogeneity in system performance. We analyze the inference latency of MobileNet [65] across hundreds of mobile phones used in a real-world FL deployment [77], and their available bandwidth. Unlike the homogeneous setting in datacenter ML, system performance across clients exhibits an order-of-magnitude difference in both computational capabilities (Figure 2(a)) and network bandwidth (Figure 2(b)).

Enormous population and pervasive uncertainty. While traditional ML runs in a well-managed cluster with a number of machines, federated learning often involves up to millions of clients, making it challenging for the coordinator to efficiently identify and manage valuable participants. During execution, devices often vary in system performance [18,40] – they may slow down or drop out – and the model performance varies in FL training as the model updates over rounds.

Privacy concerns. Inquiring about the privacy-sensitive information of clients (e.g., raw data or even data distribution) can alienate participants in contributing to FL [24, 66, 67].

[3]We report the pairwise deviation of categorical distributions between two clients, using the popular L1-divergence metric [55].

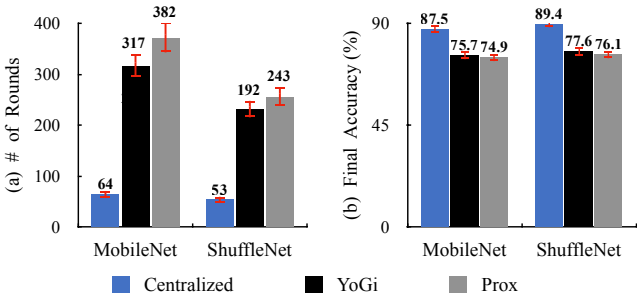


Figure 3: ***Existing works are suboptimal in: (a) round-to-accuracy performance and (b) final model accuracy. (a) reports number of rounds required to reach the highest accuracy of Prox on MobileNet (i.e., 74.9%). Error bars show standard deviation.***

Hence, realistic FL solutions have to seek efficiency improvements but with limited information available in practical FL, and their deployments must be non-intrusive to clients.

2.3 Limitations of Existing FL Solutions

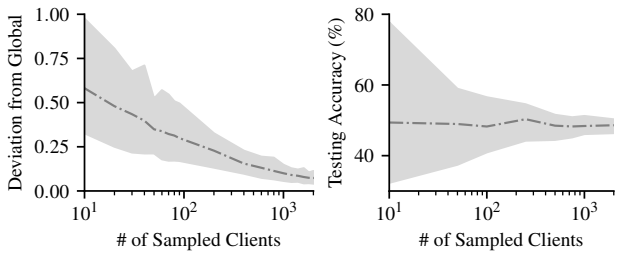
While existing FL solutions have made considerable progress in tackling some of the above challenges (§8), they mostly rely on hindsight – given a pool of participants, they optimize model performance [48,59] or system efficiency [54] to tackle data and system heterogeneity. However, the potential for curbing these disadvantages by cherry-picking participants before execution has largely been overlooked. For example, FL training and testing today still rely on randomly picking participants [18], which leaves large room for improvements.

Suboptimality in maximizing efficiency. We first show that today's participant selection underperforms for FL solutions. Here, we train two popular image classification models tailored for mobile devices (i.e., MobileNet [65] and ShuffleNet [80]) with 1.6 million images of the OpenImage dataset, and randomly pick 100 participants out of more than 14k clients in each training round. We consider a performance *upper bound* by creating a hypothetical centralized case where images are evenly distributed across only 100 clients, and train on all 100 clients in each round. As shown in Figure 3, even with state-of-the-art optimizations, such as YoGi [63] and Prox [47],[4] the round-to-accuracy and final model accuracy are both far from the upper-bound. Moreover, overlooking the system heterogeneity can elongate each round, further exacerbating the suboptimality of time-to-accuracy performance.

Inability to enforce data selection criteria. While an FL developer often fine-tunes her model by understanding the input dataset, existing solutions do not provide any systems support for her to express and reason about what data her FL model was trained or tested on. Even worse, existing participant selection not only inflates the execution, but can lead to bias and loss of confidence in results [20, 34].

To better understand how existing works fall short, we

[4]These two adapt traditional stochastic gradient descent algorithms to tackle the heterogeneity of the client datasets.



(a) Data deviation vs. participant size. (b) Accuracy vs. participant size.

Figure 4: ***Participant selection today leads to (a) deviations from developer requirements, and thus (b) affects testing result. Shadow indicates the [min, max] range of y-axis values over 1000 runs given the same x-axis input; each line reports the median.***

take the global categorical distribution as an example requirement, and experiment with the above pre-trained ShuffleNet model. Figure 4(a) shows that: (i) even for the same number of participants, random selection can result in noticeable data deviations from the target distribution; (ii) while this deviation decreases as more participants are involved, it is non-trivial to quantify how it varies with different number of participants, even if we ignore the cost of enlarging the participant set. Worse, when even selecting many participants, developers can not enforce other distributions (e.g., balanced distribution for debugging [15]) with random selection. One natural effect of violating developer specification is bias in results (Figure 4(b)), where we test the accuracy of the same model on these participants. We observe that a biased testing set results in high uncertainties in testing accuracy.

3 Oort Overview

Oort improves FL training and testing performance by judiciously selecting participants while enabling FL developers to specify data selection criteria. In this section, we provide an overview of how Oort fits in the FL life cycle to help the reader follow the subsequent sections.

3.1 Architecture

At its core, Oort is a participant selection framework that identifies and cherry-picks valuable participants for FL training and testing. It is located inside the coordinator of an FL framework and interacts with the driver of an FL execution (e.g., PySyft [7] or Tensorflow Federated [11]). Given developer-specified criteria, it responds with a list of participants, whereas the driver is in charge of initiating and managing execution on the Oort-selected remote participants.

Figure 5 shows how Oort interacts with the developer and FL execution frameworks. ① *Job submission*: the developer submits and specifies the participant selection criteria to the FL coordinator in the cloud. ② *Participant selection*: the coordinator enquires the clients meeting eligibility properties (e.g., battery level), and forwards their characteristics (e.g., liveness) to Oort. Given the developer requirements (and execution feedbacks in case of training ②a), Oort selects participants based on the given criteria and notifies the coordinator of this participant selection (②b). ③ *Execution*: the coordinator distributes relevant profiles (e.g., model) to these participants, and then each participant independently computes results (e.g., model weights in training) on her data; ④ *Aggregation*: when participants complete the computation, the coordinator aggregates updates from participants.

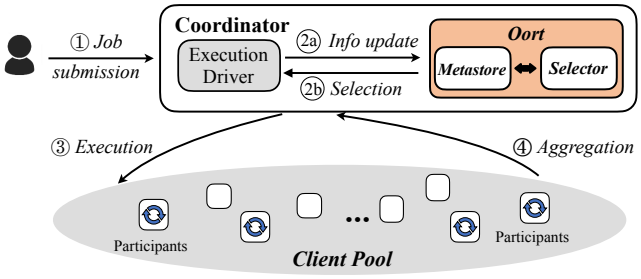


Figure 5: ***Oort architecture. The driver of the FL framework interacts with Oort using a client library.***

During federated training, where the coordinator initiates the next training round after aggregating updates from enough number of participants [18], it iterates over ②-④ in each round. Every few training rounds, federated testing is often used to detect whether the cut-off accuracy has been reached.

3.2 Oort Interface

Oort employs two distinct selectors that developers can access via a client library during FL training and testing.

Training selector. This selector aims to improve the time-to-accuracy performance of federated training. To this end, it captures the utility of clients in training, and efficiently explores and selects high-utility clients at runtime.

```
1  import Oort
2
3  def federated_model_training():
4    selector = Oort.create_training_selector(config)
5
6    # Train to target testing accuracy
7    while federated_model_testing() < target:
8
9      # Train 50 rounds before testing
10     for _ in range(50):
11       # Collect feedbacks of last round
12       feedbacks = engine.get_participant_feedback()
13
14       # Update the utility of clients
15       for clientId in feedbacks:
16         selector.update_client_util(
17                     clientId, feedbacks[clientId])
18
19       # Pick 100 high-utility participants
20       participants = selector.select_participant(100)
21       ... # Activate training on remote clients
```

Figure 6: ***Code snippet of Oort interaction during FL training.***

Figure 6 presents an example of how FL developers and frameworks interact with Oort during training. In each training round, Oort collects feedbacks from the engine driver, and updates the utility of individual clients (Line 15-17). Thereafter, it cherry-picks high-utility clients to feed the underlying execution (Line 20). We elaborate more on client utility and

the selection mechanism in Section 4.

Testing selector. This selector currently supports two types of selection criteria. When the individual client data characteristics (e.g., categorical distribution) are not provided, the testing selector determines the number of participants needed to cap the data deviation of participants from the global. Otherwise, it cherry-picks participants to serve the exact developer-specified requirements on data while minimizing the duration of testing. We elaborate more on selection for federated testing in Section 5.

4 Federated Model Training

In this section, we first outline the trade-off in selecting participants for FL training (§4.1), and then describe how Oort quantifies the client utility while respecting privacy (§4.2 and §4.3), how it selects high-utility clients at scale despite staleness in client utility as training evolves (§4.4).

4.1 Tradeoff Between Statistical and System Efficiency

Time-to-accuracy performance of FL training relies on two aspects: (i) *statistical efficiency*: the number of rounds taken to reach target accuracy; and (ii) *system efficiency*: the duration of each training round. The data stored on the client and the speed with which it can perform training determine its utility with respect to statistical and system efficiency, which we respectively refer to as statistical and system utility.

Due to the coupled nature of client data and system performance, cherry-picking participants for better time-to-accuracy performance requires us to jointly consider both forms of efficiency. We visualize the trade-off between these two with our breakdown experiments on the MobileNet model with OpenImage dataset (§7.2.1). As shown in Figure 7, while optimizing the system efficiency ("Opt-Sys. Efficiency") can reduce the duration of each round (e.g., picking the fastest clients), it can lead to more rounds than random selection as that client data may have already been overrepresented by other participants over past rounds. On the other hand, using a client with high statistical utility ("Opt-Stat. Efficiency") may lead to longer rounds if that client turns out to be the system bottleneck in global model aggregation.

Challenges. To improve time-to-accuracy performance, Oort aims to find a sweet spot in the trade-off by associating with every client its *utility* toward optimizing each form of efficiency (Figure 7). This leads to three challenges:

- In each round, how to determine which clients' data would help improve the statistical efficiency of training the most while respecting client privacy (§4.2)?
- How to take a client's system performance into account to optimize the global system efficiency (§4.3)?
- How to account for the fact that we don't have up-to-date utility values for all clients during training (§4.4)?

Next, we integrate system designs with ML principles to tackle the heterogeneity, the massive scale, the runtime uncertainties and privacy concerns of clients for practical FL.

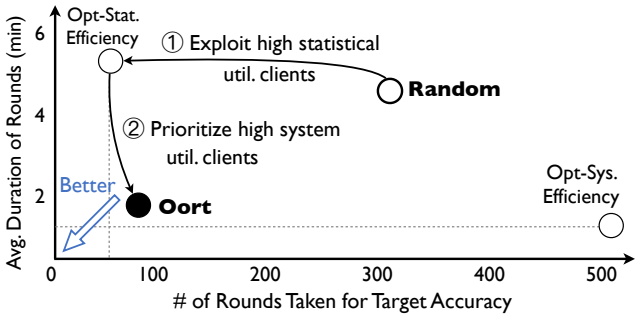


Figure 7: ***Existing FL training randomly selects participants, whereas Oort navigates the sweet point of statistical and system efficiency to optimize their circled area (i.e., time to accuracy). Numbers are from the MobileNet on OpenImage dataset (§7.2.1).***

4.2 Client Statistical Utility

An ideal design of statistical utility should be able to efficiently capture the client data utility toward improving model performance for various training tasks, and respect privacy.

To this end, we leverage importance sampling used in the ML literature [41, 81]. Say each client i has a bin B_i of training samples locally stored. Then, to improve the round-to-accuracy performance via importance sampling, the optimal solution would be to pick bin B_i with a probability proportional to its importance $|B_i|\sqrt{\frac{1}{|B_i|}\sum_{k\in B_i} \| \nabla f(k) \|^2}$, where $\| \nabla f(k) \|$ is the L2-norm of the unique sample k's gradient $\nabla f(k)$ in bin B_i. Intuitively, this means selecting the bin with larger aggregate gradient norm across all of its samples.

However, taking this importance as the statistical utility is impractical, since it requires an extra time-consuming pass over the client data to generate the gradient norm of every sample,[5] and this gradient norm varies as the model updates.

To avoid extra cost, we introduce a pragmatic approximation of statistical utility instead. At its core, the gradient is derived by taking the derivative of training loss with respect to current model weights, wherein training loss measures the estimation error between model predictions and the ground truth. Our insight is that a larger gradient norm often attributes to a bigger loss [39]. Therefore, we define the statistical utility $U(i)$ of client i as $U(i) = |B_i|\sqrt{\frac{1}{|B_i|}\sum_{k\in B_i} Loss(k)^2}$, where the training loss $Loss(k)$ of sample k is automatically generated during training with negligible collection overhead. As such, we consider clients that currently accumulate a bigger loss to be more important for future rounds.

Our statistical utility can capture the heterogeneous data utility across and within categories and samples for various tasks. We present the theoretical proof for its effectiveness over random sampling in our technical report [45], and empirically show its close-to-optimal performance (§7.2.2).

[5] ML models generate the training loss of each sample during training, but calculate the gradient of the mini-batch instead of individual samples.

How Oort respects privacy? Training loss measures the prediction confidence of a model without revealing the raw data and is often collected in real FL deployments [30, 77]. We further provide three ways to respect privacy. First, we rely on *aggregate* training loss, which is computed locally by the client across *all* of her samples without revealing the loss distribution of individual samples either. Second, when even the aggregate loss raises a privacy concern, clients can add noise to their loss value before uploading, similar to existing local differential privacy [27]. Third, we later show that Oort can flexibly accommodate other definitions of statistical utility used in our generic participant selection framework (§4.4). We provide detailed theoretical analyses for each strategy (e.g., using gradient norm of batches) of how Oort can respect privacy (e.g., amenable under noisy utility value) in our technical report [45], while empirically showing its superior performance even under noisy utility value (§7.2.3).

4.3 Trading off Statistical and System Efficiency

Simply selecting clients with high statistical utility can hamper the system efficiency. To reconcile the demand for both efficiencies, we should maximize the statistical utility we can achieve per unit time (i.e., the division of statistical utility and its round duration). As such, we formulate the utility of client i by associating her statistical utility with a global system utility in terms of the duration of each training round:

$$Util(i) = \underbrace{|B_i|\sqrt{\frac{1}{|B_i|}\sum_{k\in B_i} Loss(k)^2}}_{\text{Statistical utility } U(i)} \times \underbrace{\left(\frac{T}{t_i}\right)^{\mathbb{1}(T<t_i)\times\alpha}}_{\text{Global sys utility}} \quad (1)$$

where T is the developer-preferred duration of each round, t_i is the amount of time that client i takes to process the training, which has already been collected by today's coordinator from past rounds,[6] and $\mathbb{1}(x)$ is an indicator function that takes value 1 if x is true and 0 otherwise. This way, the utility of those clients who may be the bottleneck of the desired speed of current round will be penalized by a developer-specified factor α, but we do not reward the non-straggler clients because their completions do not impact the round duration.

This formulation assumes that all samples at a client are processed in that training round. Even if the estimated t_i for a client is greater than the desired round duration T, Oort might pick that client if the statistical utility outweighs its slow speed. Alternatively, if the developer wishes to cap every round at a certain duration [54], then either only clients with $t_i < T$ can be considered (e.g., by setting $\alpha \to \infty$) or a subset of a participant's samples can be processed [47, 63], and only the aggregate training loss of those trained data in that round is considered in measuring the statistical utility.

[6] We only care whether a client can complete by the expected duration T. So, a client can even mask its precise speed by deferring its report.

Navigating the trade-off. Determining the preferred round duration T in Equation (1), which strikes the trade-off between the statistical and system efficiency in aggregations, is non-trivial. Indeed, the total statistical utility (i.e., $\sum U(i)$) achieved by picking high utility clients can decrease round by round, because the training loss decreases as the model improves over time. If we persist in suppressing clients with high statistical utility but low system speed, the model may converge to suboptimal accuracy (§7.2.2).

To navigate the optimal trade-off – maximizing the total statistical utility achieved without greatly sacrificing the system efficiency – Oort employs a pacer to determine the preferred duration T at runtime. The intuition is that, when the accumulated statistical utility in the past rounds decreases, the pacer allows a larger $T \leftarrow T + \Delta$ by Δ to bargain with the statistical efficiency again. We elaborate more in Algorithm 1.

4.4 Adaptive Participant Selection

Given the above definition of client utility, we need to address the following practical concerns in order to select participants with the highest utility in each training round.

- *Scalability*: a client's utility can only be determined after it has participated in training; how to choose from clients at scale without having to try all clients once?
- *Staleness*: since not every client participates in every round, how to account for the change in a client's utility since its last participation?
- *Robustness*: how to be robust to outliers in the presence of corrupted clients (e.g., with noisy data)?

To tackle these challenges, we develop an exploration-exploitation strategy for participant selection (Algorithm 1).

Online exploration-exploitation of high-utility clients. Selecting participants out of numerous clients can be modeled as a multi-armed bandit problem, where each client is an "arm" of the bandit, and the utility obtained is the "reward" [14]. In contrast to sophisticated designs (e.g., reinforcement learning), the bandit model is scalable and flexible even when the solution space (e.g., number of clients) varies dramatically over time. Next, we adaptively balance the exploration and exploitation of different arms to maximize the long-term reward.

Similar to the bandit design, Oort efficiently explores potential participants under spatial variation, while intelligently exploiting observed high-utility participants under temporal variation. At the beginning of each selection round, Oort receives the feedback of the last training round, and updates the statistical utility and system performance of clients (Line 6). For the explored clients, Oort calculates their client utility and narrows down the selection by exploiting the high-utility participants (Line 9-15). Meanwhile, Oort samples $\varepsilon(\in [0, 1])$ fraction of participants to explore potential participants that had not been selected before (Line 16), which turns to full exploration as $\varepsilon \to 1$. Although we cannot learn the statistical

Input: Client set $\mathbb{C}$, sample size K, exploitation factor ε, pacer step Δ, step window W, penalty α
Output: Participant set $\mathbb{P}$

/* Initialize global variables. */
$\mathbb{E} \leftarrow \emptyset$; $\mathbb{U} \leftarrow \emptyset$ ▷ Explored clients and statistical utility.
$\mathbb{L} \leftarrow \emptyset$; $\mathbb{D} \leftarrow \emptyset$ ▷ Last involved round and duration.
$R \leftarrow 0$; $T \leftarrow \Delta$ ▷ Round counter and preferred round duration.

/* Participant selection for each round. */
Function SelectParticipant ($\mathbb{C}$, K, ε, T, α)
  $Util \leftarrow \emptyset$; $R \leftarrow R+1$
  /* Update and clip the feedback; blacklist outliers. */
  UpdateWithFeedback($\mathbb{E}$, $\mathbb{U}$, $\mathbb{L}$, $\mathbb{D}$)
  /* Pacer: Relaxes global system preference T if the statistical utility achieved decreases in last W rounds. */
  if $\sum \mathbb{U}(R-2W : R-W) > \sum \mathbb{U}(R-W : R)$ **then**
    $T \leftarrow T+\Delta$
  /* Exploitation #1: Calculate client utility. */
  for *client* $i \in \mathbb{E}$ **do**
    $Util(i) \leftarrow \mathbb{U}(i) + \sqrt{\frac{0.1 \log R}{\mathbb{L}(i)}}$ ▷ Temporal uncertainty.
    if $T < \mathbb{D}(i)$ **then** ▷ Global system utility.
      $Util(i) \leftarrow Util(i) \times (\frac{T}{\mathbb{D}(i)})^{\alpha}$
  /* Exploitation #2: admit clients with greater than c% of cut-off utility; then sample $(1-\varepsilon)K$ clients by utility. */
  $Util \leftarrow$ SortAsc(Util)
  $\mathbb{W} \leftarrow$ CutOffUtil($\mathbb{E}$, $c \times Util((1-\varepsilon) \times K)$)
  $\mathbb{P} \leftarrow$ SampleByUtil($\mathbb{W}$, $Util$, $(1-\varepsilon) \times K$)
  /* Exploration: sample unexplored clients by speed. */
  $\mathbb{P} \leftarrow \mathbb{P} \cup$ SampleBySpeed($\mathbb{C} - \mathbb{E}$, $\varepsilon \times K$)
  return $\mathbb{P}$

Alg. 1: Participant selection w/ exploration-exploitation.

utility of not-yet-tried clients, one can decide to prioritize the unexplored clients with faster system speed when possible (e.g., by inferring from device models), instead of performing random exploration (Line 16).

Exploitation under staleness in client utility. Oort employs two strategies to account for the dynamics in client utility over time. First, motivated by the confidence interval used to measure the uncertainty in bandit reward, we introduce an incentive term, which shares the same shape of the confidence in bandit solutions [37], to account for the staleness (Line 10), whereby we gradually increase the utility of a client if she has been overlooked for a long time. So those clients accumulating high utility since their last trial can still be repurposed again. Second, instead of picking clients with top-k utility deterministically, we allow a confidence interval c on the cut-off utility (95% by default in Line 13-14). Namely, we admit clients whose utility is greater than the c% of the top $((1-\varepsilon) \times K)$-th participant. Among this high-utility pool, Oort samples participants with probability proportional to their utility (Line 15). This adaptive exploitation mitigates the uncertainties in client utility by prioritizing participants opportunistically, thus relieving the need for accurate estimations of utility as we do not require the exact ordering among clients, while preserving a high quality as a whole.

```
def federated_model_testing():
  selector = Oort.create_testing_selector()

  # Type 1: subset w/ < X deviation from the global
  participants = selector.select_by_deviation(
    dev_target, range_of_capacity, total_num_clients)

  # Provide individual client data characteristics
  selector.update_client_info(client_id, client_info)
  # Type 2: [5k, 5k] samples of category [i, j]
  participants = selector.select_by_category(
                 request_list, testing_config)
```

Figure 8: ***Key Oort APIs for supporting federated testing.***

Robust exploitation under outliers. Simply prioritizing high utility clients can be vulnerable to outliers in unfavorable settings. For example, corrupted clients may have noisy data, leading to high training loss, or even report arbitrarily high training loss intentionally. For robustness, Oort (i) removes the client in selection after she has been picked over a given number of rounds. This helps to remove the perceived outliers in terms of participation (Line 6); (ii) clips the utility value of a client by capping it to no more than an upper bound (e.g., 95% value in utility distributions). With probabilistic participant selection among the high-utility client pool (Line 15), the chance of selecting outliers is significantly decreased under the scale of clients in FL. We show that Oort outperforms existing mechanisms while being robust (§7.2.3).

Accommodation to diverse selection criteria. Our adaptive participant selection is generic for different utility definitions of diverse selection criteria. For example, developers may hope to reconcile their demand for time-to-accuracy efficiency and fairness, so that some clients are not underrepresented (e.g., more fair resource usage across clients) [40, 48]. Although developers may have various fairness criterion $fairness(\cdot)$, Oort can enforce their demands by replacing the current utility definition of client i with $(1-f) \times Util(i) + f \times fairness(i)$, where $f \in [0,1]$ and Algorithm 1 will naturally prioritize clients with the largest fairness demand as $f \to 1$. For example, $fairness(i) = max_resource_usage - resource_usage(i)$ motivates fair resource usage for each client i. Note that existing participant selection provides no support for fairness, and we show that Oort can efficiently enforce diverse developer-preferred fairness while improving performance (§7.2.3).

5 Federated Model Testing

Enforcing developer-defined requirements on data distribution is a first-order goal in FL testing, whereas existing mechanisms lead to biased testing results (§2.3). In this section, we

elaborate on how Oort serves the two primary types of queries. As shown in Figure 8, we start with how Oort preserves the representativeness of testing set even without individual client data characteristics (§5.1), and how it efficiently enforces developer's testing criteria for specific data distribution when the individual information is provided (§5.2).

5.1 Preserving Data Representativeness

Learning the individual data characteristics (e.g., categorical distribution) can be too expensive or even prohibited [25, 64]. Without knowing data characteristics, the developer has to be conservative and selects many participants to gain more confidence for query *"a testing set with less than X% data deviation from the global"*, as selecting too few can lead to a biased testing result (§2.3). However, admitting too many may inflate the budget and/or take too long because of the system heterogeneity. Next, we show how Oort can enable guided participant selection by determining the number of participants needed to guarantee this deviation target.

We consider the deviation of the data formed by all participants from the global dataset (i.e., representative) using L1-distance, a popular distance metric in FL [34, 35, 57]. For category X, its L1-distance ($|\bar{X} - E[\bar{X}]|$) captures how the average number of samples of all participants (i.e., empirical value $\bar{X}$) deviates from that of all clients (i.e., expectation $E[\bar{X}]$). Note that the number of samples X_n that client n holds is independent across clients. Namely, the number of samples that one client holds will not be affected by the selection of any other clients at that time, so it can be viewed as a random instance sampled from the distribution of variable X.

Given the developer-specified tolerance ε on data deviation and confidence interval δ (95% by default [56]), our goal is to estimate the number of participants needed such that the deviation from the representative categorical distribution is bounded (i.e., $Pr[|\bar{X} - E[\bar{X}]| < \varepsilon] > \delta$). To this end, we formulate it as a problem of sampling stochastic variables, and apply the Hoeffding bound [16] to capture how this data deviation varies with different number of participants. We attach our theoretical results and proof in our technical report [45].

Estimating the number of participants to cap deviation. Even when the individual data characteristics are not available, the developer can specify her tolerance ε on the deviation from the global categorical distribution, whereby Oort outputs the number of participants needed to preserve this preference. To use our model, the developer needs to input the global range (i.e., global maximum - global minimum) of the number of samples that one client can hold, and the total number of clients. Learning this global information securely is well-established [23, 64], and the developer can assume a plausible limit (e.g., according to the capacity of device models) too.

Our model does not require any collection of the distribution of global or participant data. As a straw-man participant selection design, the developer can randomly distribute her model to this Oort-determined number of participants. After collecting results from this number of participants, she can confirm the representativeness of computed data.

5.2 Enforcing Diverse Data Distribution

When the individual data characteristics are provided (e.g., FL across enterprise AI cameras [35, 51]), Oort can enforce the exact data preference on specific categorical distribution, and improve the duration of testing by cherry-picking participants.

Satisfying queries like *"[5k, 5k] samples of class [x, y]"* can be viewed as a multi-dimensional bin covering problem, where a subset of data bins (i.e., participants) are selected to cover the requested quantity of data. For each category $i(\in I)$ of interest, the developer has preference p_i (preference constraint), and an upper limit B (referred to as budget) on how many participants she can have [15]. Each participant $n(\in N)$ can contribute n_i samples out of her capacity ${c_n}^i$ (capacity constraint). Given her compute speed s_n, the available bandwidth b_n and the size of data transfers d_n, we aim to minimize the duration of model testing:

$$\min \left\{ \max_{n \in N} \left(\frac{\sum_{i \in I} n_i}{s_n} + \frac{d_n}{b_n} \right) \right\} \quad \triangleright \text{ Minimize duration}$$

$$\text{s.t.} \quad \forall i \in I, \sum_{n \in N} n_i = p_i \quad \triangleright \text{ Preference Constraint}$$

$$\forall i \in I, \forall n \in N, n_i \leq c_n^i \quad \triangleright \text{ Capacity Constraint}$$

$$\forall i \in I, \sum_{n \in N} \mathbb{1}(n_i > 0) \leq B \quad \triangleright \text{ Budget Constraint}$$

The max-min formulation stems from the fact that testing completes after aggregating results from the last participant. While this mixed-integer linear programming (MILP) model provides high-quality solutions, it has prohibitively high computational complexity for large N.

Scalable participant selection. For better scalability, we present a greedy heuristic to scale down the search space of this strawman. We (1) first group a subset of feasible clients to satisfy the preference constraint. To this end, we iteratively add to our subset the client which has the most number of samples across all not-yet-satisfied categories, and deduct the preference constraint on each category by the corresponding capacity of this client. We stop this greedy grouping until the preference is met, or request a new budget if we exceed the budget; and (2) then optimize job duration with a simplified MILP among this subset of clients, wherein we have removed the budget constraint and reduced the search space of clients. We show that our heuristic can outperform the straw-man MILP model in terms of the end-to-end duration of model testing owing to its small overhead (§7.3.2).

6 Implementation

We have implemented Oort as a Python library, with 2617 lines of code, to friendly support FL developers. Oort provides simple APIs to abstract away the problem of participant selection, and developers can import Oort in their application

codebase and interact with FL engines (e.g., PySyft [7] or TensorFlow Federated [11]).

We have integrated Oort with PySyft. Oort operates on and updates its client metadata (e.g., data distribution or system performance) fed by the FL developer and PySyft at runtime. The metadata of each client in Oort is an object with a small memory footprint. Oort caches these objects in memory during executions and periodically backs them up to persistent storage. In case of failures, the execution driver will initiate a new Oort selector, and load the latest checkpoint to catch up. We employ Gurobi solver [5] to solve the MILP. The developer can also initiate a Oort application beyond coordinators to avoid resource contention. We use *xmlrpc* library to connect to the coordinator, and these updates will activate Oort to write these updates to its metastore. In the coordinator, we use the PySyft API *model.send(client_id)* to direct which client to run given the Oort decision, and *model.get(client_id)* to collect the feedback.

7 Evaluation

We evaluate Oort's effectiveness for four different ML models on four CV and NLP datasets. We organize our evaluation by the FL activities with the following key results.

FL training results summary:

- Oort outperforms existing random participant selection by 1.2×-14.1× in time-to-accuracy performance, while achieving 1.3%-9.8% better final model accuracy (§7.2.1).
- Oort achieves close-to-optimal model efficiency by adaptively striking the trade-off between statistical and system efficiency with different components (§7.2.2).
- Oort outperforms its counterpart over a wide range of parameters and different scales of experiments, while being robust to outliers (§7.2.3).

FL testing results summary:

- Oort can serve testing criteria on data deviation while reducing costs by bounding the number of participants needed without individual data characteristics (§7.3.1).
- With the individual information, Oort improves the testing duration by 4.7× w.r.t. Mixed Integer Linear Programming (MILP) solver, and is able to efficiently enforce developer preferences across millions of clients (§7.3.2).

7.1 Methodology

Experimental setup. Oort is designed to operate in large deployments with potentially millions of edge devices. However, such a deployment is not only prohibitively expensive, but also impractical to ensure the reproducibility of experiments. As such, we resort to a cluster with 68 NVIDIA Tesla P100 GPUs, and emulate up to 1300 participants in each round. We simulate real-world heterogeneous client system performance and data in both training and testing evaluations using an open-source FL benchmark [43]: (1) Heterogeneous device runtimes of different models, network throughput/connectivity, device model and availability are emulated using data from AI Benchmark [1] and Network Measurements on mobiles [6]; (2) We distribute each real dataset to clients following the corresponding raw placement (e.g., using <*authors_ID*> to allocate OpenImage), where client data can vary in quantities, distribution of outputs and input features; (3) The coordinator communicates with clients using the parameter server architecture. These follow the PySyft and real FL deployments. To mitigate stragglers, we employ the widely-used mechanism specified in real FL deployments [18], where we collect updates from the first K completed participants out of $1.3K$ participants in each round, and K is 100 by default. We report the simulated clock time of clients in evaluations.

Dataset	# of Clients	# of Samples
Google Speech [74]	2,618	105,829
OpenImage-Easy [3]	14,477	871,368
OpenImage [3]	14,477	1,672,231
StackOverflow [9]	315,902	135,818,730
Reddit [8]	1,660,820	351,523,459

Table 1: ***Statistics of the dataset in evaluations.***

Datasets and models. We run three categories of applications with four real-world datasets of different scales, and Table 1 reports the statistics of each dataset:

- *Speech Recognition*: the small-scale Google speech dataset [74]. We train a convolutional neural network model (ResNet-34 [31]) to recognize the command among 35 categories.
- *Image Classification*: the middle-scale OpenImage [3] dataset, with 1.5 million images spanning 600 categories, and a simpler dataset (OpenImage-Easy) with images from the most popular 60 categories. We train MobileNet [65] and ShuffleNet [80] models to classify the image.
- *Language Modeling*: the large-scale StackOverflow [9] and Reddit [8] dataset. We train next word predictions with Albert model [46] using the top-10k popular words.

These applications are widely used in real end-device applications [75], and these models are designed to be lightweight.

Parameters. The minibatch size of each participant is 16 in speech recognition, and 32 in other tasks. The initial learning rate for Albert model is 4e-5, and 0.04 for other models. These configurations are consistent with those reported in the literature [29]. In configuring the training selector, Oort uses the popular time-based exploration factor [14], where the initial exploration factor is 0.9, and decreased by a factor 0.98 after each round when it is larger than 0.2. The step window of pacer W is 20 rounds. We set the pacer step Δ in a way that it can cover the duration of next $W \times K$ clients in the descending order of explored clients' duration, and the straggler penalty α to 2. We remove a client from Oort's exploitation list once

Task	Dataset	Accuracy Target	Model	Speedup for Prox [47]			Speedup for YoGi [63]		
				Stats.	Sys.	Overall	Stats.	Sys.	Overall
Image Classification	OpenImage-Easy [3]	74.9%	MobileNet [65]	3.8×	3.2×	12.1×	2.4×	2.4×	5.7×
			ShuffleNet [80]	2.5×	3.5×	8.8×	1.9×	2.7×	5.1×
	OpenImage [3]	53.1%	MobileNet	4.2×	3.1×	13.0×	2.3×	1.5×	3.3×
			ShuffleNet	4.8×	2.9×	14.1×	1.8×	3.2×	5.8×
Language Modeling	Reddit [8]	39 perplexity	Albert [46]	1.3×	6.4×	8.4×	1.5×	4.9×	7.3×
	StackOverflow [9]	39 perplexity	Albert	2.1×	4.3×	9.1×	1.8×	4.4×	7.8×
Speech Recognition	Google Speech [74]	62.2%	ResNet-34 [31]	1.1×	1.1×	1.2×	1.2×	1.1×	1.3×

Table 2: ***Summary of improvements on time to accuracy.***[7] ***We tease apart the overall improvement with statistical and system ones, and take the highest accuracy that Prox can achieve as the target, which is moderate due to the high task complexity and lightweight models.***

she has been selected over 10 times.

Metrics. We care about the *time-to-accuracy* performance and *final model accuracy* of model training tasks on the testing set. For model testing, we measure the *end-to-end* testing duration, which consists of the computation overhead of the solution and the duration of actual computation.

For each experiment, we report the mean value over 5 runs, and error bars show the standard deviation.

7.2 FL Training Evaluation

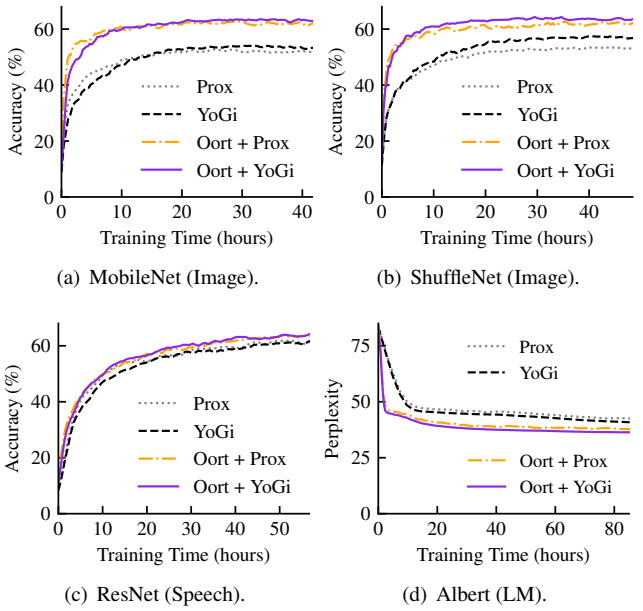
In this section, we evaluate Oort's performance on model training, and employ Prox [47] and YoGi [63]. We refer Prox as Prox running with existing random participant selection, and Prox + Oort is Prox running atop Oort. We use a similar denotation for YoGi. Note that Prox and YoGi optimize the statistical model efficiency for the given participants, while Oort cherry-picks participants to feed them.

7.2.1 End-to-End Performance

Table 2 summarizes the key time-to-accuracy performance of all datasets. In the rest of the evaluations, we report the ShuffleNet and MobileNet performance on OpenImage, and Albert performance on Reddit dataset for brevity. Figure 9 reports the timeline of training to achieve different accuracy.

Oort improves time-to-accuracy performance. We notice that Oort achieves large speedups to reach the target accuracy (Table 2). Oort reaches the target 3.3×-14.1× faster in terms of wall clock time on the middle-scale OpenImage dataset; speedup on the large-scale Reddit and StackOverflow dataset is 7.3×-9.1×. Understandably, these benefits decrease when the total number of clients is small, as shown on the small-scale Google Speech dataset (1.2×-1.3×).

[7] We set the target accuracy to be the highest achievable accuracy by all used strategies, which turns out to be Prox accuracy. Otherwise, some may never reach that target.



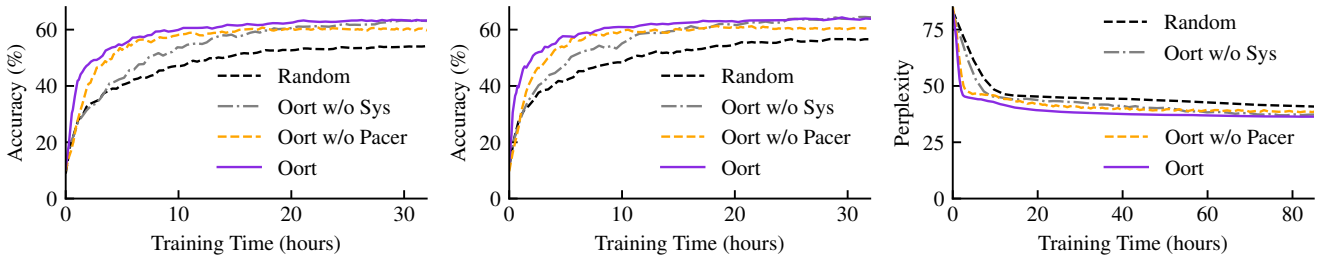
(a) MobileNet (Image). (b) ShuffleNet (Image).

(c) ResNet (Speech). (d) Albert (LM).

Figure 9: ***Time-to-Accuracy performance. A lower perplexity is better in the language modeling (LM) task.***

These time-to-accuracy improvements stem from the comparable benefits in statistical model efficiency and system efficiency (Table 2). Oort takes 1.8×-4.8× fewer training rounds on OpenImage dataset to reach the target accuracy, which is better than that of language modeling tasks (1.3×-2.1×). This is because real-life images often exhibit greater heterogeneity in data characteristics than the language dataset, whereas the large population of language datasets leaves a great potential to prioritize clients with faster system speed.

Oort improves final model accuracy. When the model converges, Oort achieves 6.6%-9.8% higher final accuracy on OpenImage dataset, and 3.1%-4.4% better perplexity on Reddit dataset (Figure 9). Again, this improvement on Google Speech dataset is smaller (1.3% for Prox and 2.2% for YoGi)



(a) MobileNet (Image). (b) ShuffleNet (Image). (c) Albert (LM).

Figure 10: ***Breakdown of Time-to-Accuracy performance with YoGi, when using different participant selection strategies.***

due to the small scale of clients. These improvements attribute to the exploitation of high statistical utility clients. Specifically, the statistical model accuracy is determined by the quality of global aggregation. Without cherry-picking participants in each round, clients with poor statistical model utility can dilute the quality of aggregation. As such, the model may converge to suboptimal performance. Instead, models running with Oort concentrate more on clients with high statistical utility, thus achieving better final accuracy.

7.2.2 Performance Breakdown

We next delve into the improvement on middle- and large-scale datasets, as they are closer to real FL deployments. We break down our knobs designed for striking the balance between statistical and system efficiency: (i) (*Oort w/o Pacer*): We disable the pacer that guides the aggregation efficiency. As such, it keeps suppressing low-speed clients, and the training can be restrained among low-utility but high-speed clients; (ii) (*Oort w/o Sys*): We further totally remove our benefits from system efficiency by setting α to 0, so Oort blindly prioritizes clients with high statistical utility. We take YoGi for analysis, because it outperforms Prox most of the time.

Breakdown of time-to-accuracy efficiency. Figure 10 reports the breakdown of time-to-accuracy performance, where Oort achieves comparable improvement from statistical and system optimizations. Taking Figure 10(b) as an example, (i) At the beginning of training, both Oort and (Oort w/o Pacer) improve the model accuracy quickly, because they penalize the utility of stragglers and select clients with higher statistical utility and system efficiency. In contrast, (Oort w/o Sys) only considers the statistical utility, resulting in longer rounds. (ii) As training evolves, the pacer in Oort gradually relaxes the constraints on system efficiency, and admits clients with relatively low speed but higher statistical utility, which ends up with the similar final accuracy of (Oort w/o Sys). However, (Oort w/o Pacer) relies on a fixed system constraint and suppresses valuable clients with high statistical utility but low speed, leading to suboptimal final accuracy.

Oort achieves close to upper-bound statistical performance. We consider an *upper-bound* statistical efficiency by creating a centralized case, where all data are evenly distributed to K participants. Using the target accuracy in Table 2, Oort can efficiently approach this upper bound by incorporating different components (Figure 11). Oort is within $2\times$ of the upper-bound to achieve the target accuracy, and (Oort w/o Sys) performs the best in statistical model efficiency, because (Oort w/o Sys) always grasps clients with higher statistical utility. However, it is suboptimal in our targeted time-to-accuracy performance because of ignoring the system efficiency. Moreover, by introducing the pacer, Oort achieves 2.4%-3.1% better accuracy than (Oort w/o Pacer), and is merely about 2.7%-3.3% worse than the upper-bound final model accuracy (Figure 12).

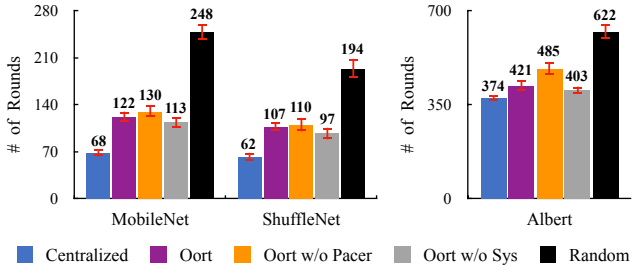


Figure 11: ***Number of rounds to reach the target accuracy.***

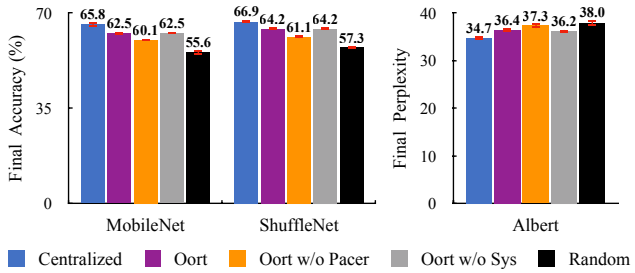
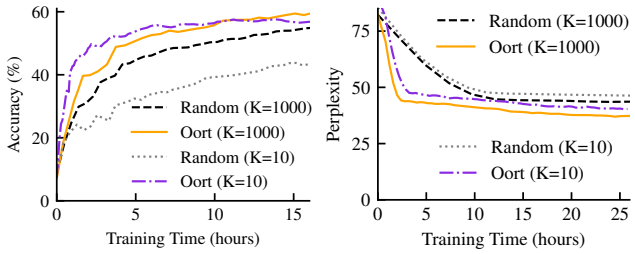


Figure 12: ***Breakdown of final model accuracy.***

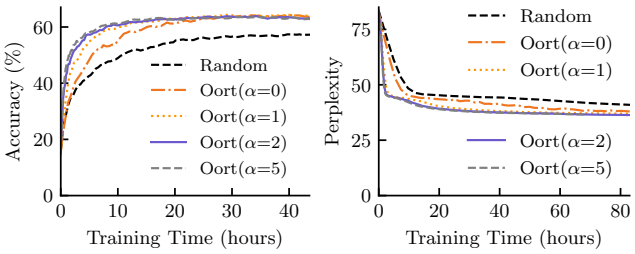
7.2.3 Sensitivity Analysis

Impact of number of participants K. We evaluate Oort across different scales of participants in each round, where we cut off the training after 200 rounds given the diminishing rewards. We observe that Oort improves time-to-accuracy efficiency across different number of participants (Figure 13), and having more participants in FL indeed receives diminishing rewards. This is because taking more participants (i) is



(a) ShuffleNet (Image). (b) Albert (LM).

Figure 13: ***Oort outperforms in different scales of participants.***



(a) ShuffleNet (Image). (b) Albert (LM).

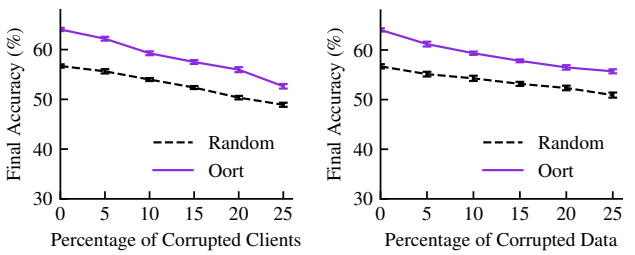
Figure 14: ***Oort improves performance across penalty factors.***

similar to having a large batch size, which is confirmed to be even negative to round-to-accuracy performance [50]; (ii) can lead to longer rounds due to stragglers when the number of clients is limited (e.g., K=1000 on OpenImage dataset).

Impact of penalty factor α on stragglers. Oort uses the penalty factor α to penalize the utility of stragglers in participant selection, whereby it adaptively prioritizes high system efficiency participants. Figure 14 shows that Oort outperforms its counterparts across different α. Note that Oort orchestrates its components to automatically navigate the best performance across parameters: larger α (i.e., overemphasizing system efficiency) drives the Pacer to relax the system constraint T more frequently to admit clients with higher statistical efficiency, and vice versa. As such, Oort achieves similar performance across all non-zero α.

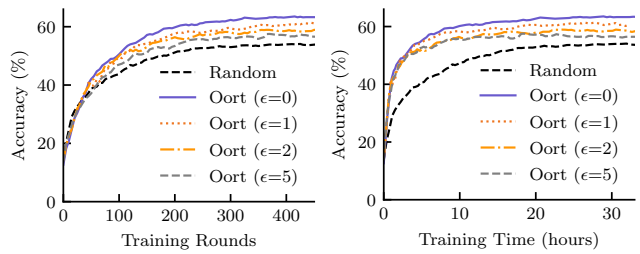
Impact of outliers. We investigate the robustness of Oort by introducing outliers manually. Following the popular adversarial ML setting [26], we randomly flip the ground-truth data labels of the OpenImage dataset to any other categories, resulting in artificially high utility. We consider two practical scenarios with the ShuffleNet model: (i) Corrupted clients: labels of all training samples on these clients are flipped (Figure 15(a)); (ii) Corrupted data: each client uniformly flips a subset of her training samples (Figure 15(b)). We notice Oort still outperforms across all degrees of corruption.

Impact of noisy utility. We next show the superior performance of Oort over its counterparts under noisy utility value. In this experiment, we add noise from the Gaussian distribution $Gaussian(0,\sigma^2)$, and investigate Oort's performance with different σ. Similar to differential FL [27], we

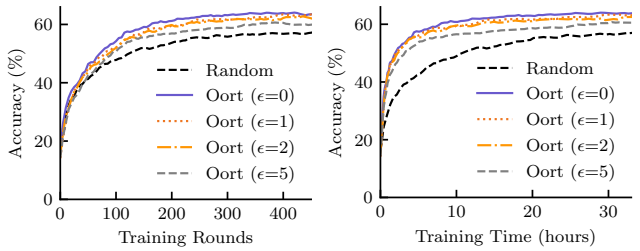


(a) Corrupted clients. (b) Corrupted data.

Figure 15: ***Oort still improves performance under outliers.***



(a) Round to accuracy (MobileNet). (b) Time to accuracy (MobileNet).



(c) Round to accuracy (ShuffleNet). (d) Time to accuracy (ShuffleNet).

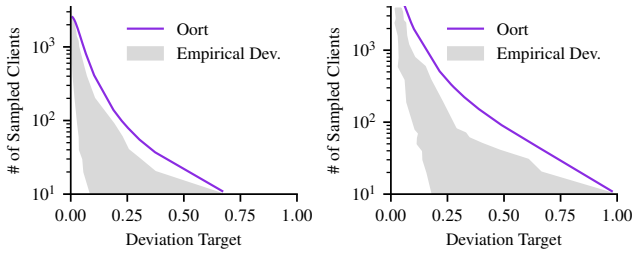
Figure 16: ***Oort improves performance even under noise.***

define $\sigma = \epsilon \times Mean(real_value)$, where $Mean(real_value)$ is the average real value without noise. Note that we take this $real_value$ as reference for the ease of presentations, and developers can refer to other values. As such, a large ϵ implies larger variance in noise, thus providing better privacy by disturbing the real value significantly. We report the statistical efficiency after adding noise to the statistical utility (Fig 16(a) and Fig 16(c)), as well as the time-to-accuracy performance (Fig 16(b) and Fig 16(d)). We observe that Oort still improves performance across different amount of noise, and is robust even when the noise is large (e.g., $\epsilon = 5$ is often considered to be very large noise [12]).

Oort can respect developer-preferred fairness. In this experiment, we expect all clients should have participated training with the same number of rounds (Table 3), implying a fair resource usage [40]. We train ShuffleNet model on OpenImage dataset with YoGi. To this end, we sweep different knobs f to accommodate the developer demands for the time-to-accuracy efficiency and fairness. Namely, we replace the current utility definition of client i with $(1-f) \times Util(i) + f \times fairness(i)$, where $fairness(i) =$

Strategy	TTA (h)	Final Accuracy (%)	Var. (Rounds)
Random	36.3	57.3	0.39
$f = 0$	5.8	64.2	6.52
$f = 0.25$	6.1	62.4	5.1
$f = 0.5$	13.1	59.7	2.03
$f = 0.75$	25.4	58.6	0.65
$f = 1$	30.1	57.2	0.31

Table 3: ***Oort improves time to accuracy (TTA) across different fairness knobs (f). Random reports the performance of random participant selection. The variance of rounds reports how fairness is enforced in terms of the number of participating rounds across clients. A smaller variance implies better fairness.***



(a) Google Speech. (b) Reddit.

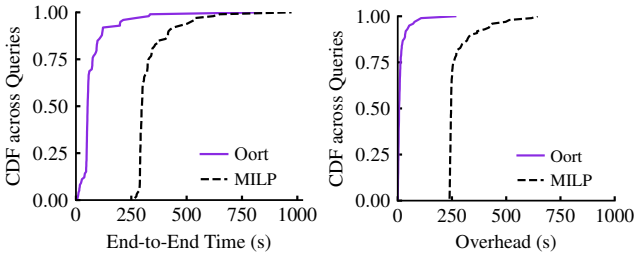
Figure 17: ***Oort can cap data deviation for all targets. Shadow indicates the empirical [min, max] range of the x-axis values over 1000 runs given the y-axis input.***

$max_resource_usage - resource_usage(i)$. Understandably, time-to-accuracy efficiency significantly decreases as $f \to 1$, since we gradually end up with round-robin participant selection, totally ignoring the utility of clients. Note that Oort still achieves better time-to-accuracy even when $f \to 1$ as it prioritizes high system utility clients at the beginning of training, thus achieving shorter rounds. Moreover, Oort can enforce different fairness preferences while improving efficiency across fairness knobs.

7.3 FL Testing Evaluation

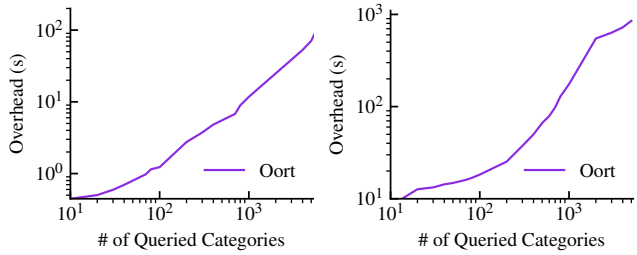
7.3.1 Preserving Data Representativeness

Oort can cap data deviation. Figure 17 reports Oort's performance on serving different deviation targets, with respect to the global distribution. We sweep the number of selected clients from 10 to 4k, and randomly select each given number of participants over 1k times to empirically search their possible deviation. We notice that for a given deviation target, (i) different workloads require distinct number of participants. For example, to meet the target of 0.05 divergence, the Speech dataset uses 6× less participants than the Reddit attributing to its smaller heterogeneity (e.g., tighter range of the number of samples); (ii) with the Oort-determined number of participants, no empirical deviation exceeds the target, showing the effectiveness of Oort in satisfying the deviation target, whereby Oort reduces the cost of expanding participant set arbitrarily and improves the testing duration.



(a) OpenImage (Testing duration). (b) OpenImage (Overhead).

Figure 18: ***Oort outperforms MILP in clairvoyant FL testing.***



(a) StackOverflow (0.3M clients). (b) Reddit (1.6M clients).

Figure 19: ***Oort scales to millions of clients, while MILP did not complete on any query.***

7.3.2 Enforcing Diverse Data Distribution

Oort outperforms MILP. We start with the middle-scale OpenImage dataset and compare the end-to-end testing duration of Oort and MILP. Here, we generate 200 queries using the form "Give me X representative samples", where we sweep X from 4k to 200k and budget B from 100 participants to 5k participants. We report the validation time of MobileNet on participants selected by these strategies.

Figure 18(a) shows the end-to-end testing duration. We observe Oort outperforms MILP by 4.7× on average. This is because Oort suffers little computation overhead by greedily reducing the search space of MILP. As shown in Figure 18(b), MILP takes 274 seconds on average to complete the participant selection, while Oort only takes 15 seconds.

Oort is scalable. We further investigate Oort's performance on the large-scale StackOverflow and Reddit dataset with millions of clients, where we take 1% of the global data as the requirement, and sweep the number of interested categories from 1 to 5k. Figure 19 shows even though we gradually magnify the search space of participant selection by introducing more categories, Oort can serve our requirement in a few minutes at the scale of millions of clients, while MILP fails to generate the solution decision for any query.

8 Related Work

Federated Learning Federated learning [40] is a distributed machine learning paradigm in a network of end devices, wherein Prox [47] and YoGi [63] are state-of-the-art optimizations in tackling data heterogeneity. Recent efforts in FL have been focusing on improving communication effi-

ciency [33,54] or compression schemes [13], ensuring privacy by leveraging multi-party computation (MPC) [19] and differential privacy [27], or tackling heterogeneity by reinventing ML algorithms [48,72]. However, they underperform in FL because of the suboptimal participant selection they rely on, and lack systems supports for developers to specify their participant selection criteria.

Datacenter Machine Learning Distributed ML in datacenters has been well-studied [36,58,60], wherein they assume relatively homogeneous data and workers [28,52]. While developer requirements and models can still be the same, the heterogeneity of client system performance and data distribution makes FL much more challenging. We aim at enabling them in FL. To accelerate traditional model training, some techniques bring up importance sampling to prioritize important training samples in selecting mini-batches for training [39,41,81]. While bearing some resemblance in prioritizing data, Oort adaptively considers both statistical and system efficiency in formulating the client utility at scale.

Geo-distributed Data Analytics Federated data analytics has been a topic of interest in geo-distributed storage [69] and data processing systems [44,79] that attempt to reduce latency [70] and/or save bandwidth [42,61,71]. Gaia [33] reduces network traffics for model training across datacenters, while Sol [44] enables generic federated computation on data with sub-second latency in the execution layer. These work back up Oort with cross-layer system support, whereas Oort cherry-picks participants before execution.

Privacy-preserving Data Analytics To gather sensitive statistics from user devices, several differentially private systems add noise to user inputs locally to ensure privacy [25], but this can reduce the accuracy. Some assume a trusted third party, which only adds noise to the aggregated raw inputs [17], or use MPC to enable global differential privacy without a trusted party [64]. While our goal is not to address the security and privacy issue in these solutions, Oort enables informed participant selection by leveraging the information already available in today's FL, and can reconcile with them (e.g., to deliver improvement under outliers while respecting privacy).

9 Conclusion

While today's FL efforts have been optimizing the statistical model and system efficiency by reinventing traditional ML designs, the participant selection mechanisms they rely on underperform for federated training and testing, and fail to enforce diverse data selection criteria. In this paper, we present Oort to enable guided participant selection for FL developers. Compared to existing mechanisms, Oort achieves large speedups in time-to-accuracy performance for federated training by picking clients with high statistical and system utility, and it allows developers to specify their selection criteria on data while efficiently serving their requirements on data distribution during testing even at the scale of millions of clients. The artifacts of Oort are available at https://github.com/SymbioticLab/Oort.

Acknowledgments

Special thanks go to the entire ConFlux team and CloudLab team for making Oort experiments possible. We would also like to thank the anonymous reviewers, our shepherd, Gennady Pekhimenko, and SymbioticLab members for their insightful feedback. This work was supported in part by NSF grants CNS-1900665 and CNS-1909067.

References

[1] AI Benchmark: All About Deep Learning on Smartphones. http://ai-benchmark.com/ranking_deeplearning_detailed.html.

[2] Federated AI Technology Enabler. https://www.fedai.org/.

[3] Google Open Images Dataset. https://storage.googleapis.com/openimages/web/index.html.

[4] Google's Sundar Pichai: Privacy Should Not Be a Luxury Good. https://www.nytimes.com/2019/05/07/opinion/google-sundar-pichai-privacy.html.

[5] Gurobi. https://www.gurobi.com/.

[6] MobiPerf. https://www.measurementlab.net/tests/mobiperf/.

[7] PySyft. https://github.com/OpenMined/PySyft.

[8] Reddit Comment Data. https://files.pushshift.io/reddit/comments/.

[9] Stack Overflow Data. https://cloud.google.com/bigquery/public-data/stackoverflow.

[10] Stanford Puffer. https://puffer.stanford.edu/.

[11] TensorFlow Federated. https://www.tensorflow.org/federated.

[12] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *CCS*, 2016.

[13] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. QSGD: Communication-efficient sgd via gradient quantization and encoding. In *NeurIPS*, 2017.

[14] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. In *Machine Learning*, 2002.

[15] Sean Augenstein, H Brendan McMahan, Daniel Ramage, Swaroop Ramaswamy, Peter Kairouz, Mingqing Chen, Rajiv Mathews, et al. Generative models for effective ML on private, decentralized datasets. In *ICLR*, 2020.

[16] Rémi Bardenet and Odalric-Ambrym Maillard. *Concentration inequalities for sampling without replacement*. Bernoulli Society for Mathematical Statistics and Probability, 2015.

[17] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnes, and Bernhard Seefeld. Prochlo: Strong privacy for analytics in the crowd. In *SOSP*, 2017.

[18] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečnỳ, Stefano Mazzocchi, H Brendan McMahan, et al. Towards federated learning at scale: System design. In *MLSys*, 2019.

[19] Keith Bonawitz, Vladimir Ivanov, and et al. Practical secure aggregation for privacy-preserving machine learning. In *CCS*, 2017.

[20] Eric Breck, Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data validation for machine learning. In *MLSys*, 2019.

[21] Mingqing Chen, Rajiv Mathews, Tom Ouyang, and Françoise Beaufays. Federated learning of out-of-vocabulary words. In *arxiv.org/abs/1903.10635*, 2019.

[22] Mingqing Chen, Ananda Theertha Suresh, Rajiv Mathews, Adeline Wong, Cyril Allauzen, Francoise Beaufays, and Michael Riley. Federated learning of n-gram language models. In *ACL*, 2019.

[23] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *NSDI*, 2017.

[24] Apple Differential Privacy Team. Learning with privacy at scale. In *Apple Machine Learning Journal*, 2017.

[25] Ulfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. RAPPOR: Randomized aggregatable privacy-preserving ordinal response. In *CCS*, 2014.

[26] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[27] Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially private federated learning: A client level perspective. In *NeuIPS*, 2017.

[28] Juncheng Gu, Mosharaf Chowdhury, Kang G Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *NSDI*, 2019.

[29] Andrew Hard, Kanishka Rao, Rajiv Mathews, Swaroop Ramaswamy, Françoise Beaufays, Sean Augenstein, Hubert Eichner, Chloé Kiddon, and Daniel Ramage. Federated learning for mobile keyboard prediction. In *arxiv.org/abs/1811.03604*, 2018.

[30] Florian Hartmann, Sunah Suh, Arkadiusz Komarzewski, Tim D. Smith, and Ilana Segall. Federated learning for ranking browser history suggestions. In *arxiv.org/abs/1911.11807*, 2019.

[31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[32] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B Gibbons, and Onur Mutlu. Focus: Querying large video datasets with low latency and low cost. In *OSDI*, 2018.

[33] Kevin Hsieh, Aaron Harlap, Nandita Vijaykumar, Dimitris Konomis, Gregory R. Ganger, Phillip B. Gibbons, and Onur Mutlu. Gaia: Geo-distributed machine learning approaching LAN speeds. In *NSDI*, 2017.

[34] Kevin Hsieh, Amar Phanishayee, Onur Mutlu, and Phillip B. Gibbons. The Non-IID data quagmire of decentralized machine learning. In *ICML*, 2020.

[35] Harry Hsu, Hang Qi, and Matthew Brown. Federated visual classification with real-world data distribution. In *ECCV*, 2020.

[36] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *SOSP*, 2019.

[37] Junchen Jiang, Rajdeep Das, Ganesh Ananthanarayanan, Philip A Chou, Venkata Padmanabhan, Vyas Sekar, Esbjorn Dominique, Marcin Goliszewski, Dalibor Kukoleca, Renat Vafin, et al. Via: Improving internet telephony call quality using predictive relay selection. In *SIGCOMM*, 2016.

[38] Junchen Jiang, Yuhao Zhou, Ganesh Ananthanarayanan, Yuanchao Shu, and Andrew A. Chien. Networked cameras are the new big data clusters. In *HotEdgeVideo*, 2019.

[39] Tyler B. Johnson and Carlos Guestrin. Training deep models faster with robust, approximate importance sampling. In *NeurIPS*, 2018.

[40] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Keith Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. In *Foundations and Trends in Machine Learning*, 2021.

[41] Angelos Katharopoulos and Francois Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *ICML*, 2018.

[42] Fan Lai, Mosharaf Chowdhury, and Harsha Madhyastha. To relay or not to relay for inter-cloud transfers? In *10th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 18)*, Boston, MA, July 2018. USENIX Association.

[43] Fan Lai, Yinwei Dai, Xiangfeng Zhu, and Mosharaf Chowdhury. FedScale: Benchmarking model and system performance of federated learning. In *arxiv.org/abs/2105.11367*, 2021.

[44] Fan Lai, Jie You, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. Sol: Fast distributed computation over slow networks. In *NSDI*, 2020.

[45] Fan Lai, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. Oort: Efficient federated learning via guided participant selection. In *arxiv.org/abs/2010.06081*, 2020.

[46] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *ICLR*, 2020.

[47] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *MLSys*, 2020.

[48] Tian Li, Manzil Zaheer, Ahmad Beirami, and Virginia Smith. Fair resource allocation in federated learning. In *ICLR*, 2020.

[49] Wenqi Li, Fausto Milletari, and Daguang Xu. Privacy-preserving federated brain tumour segmentation. In *Machine Learning in Medical Imaging*, 2019.

[50] Tao Lin, Sebastian U. Stich, Kumar Kshitij Patel, and Martin Jaggi. Don't use large mini-batches, use local SGD. In *ICLR*, 2020.

[51] Jiahuan Luo, Xueyang Wu, Yun Luo, Anbu Huang, Yunfeng Huang, Yang Liu, and Qiang Yang. Real-world image datasets for federated learning. In *arxiv.org/abs/1910.11089*, 2019.

[52] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *NSDI*, 2020.

[53] Yishay Mansour, Mehryar Mohri, Jae Ro, and Ananda Suresh. Three approaches for personalization with applications to federated learning. In *arxiv.org/abs/2002.10619*, 2020.

[54] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.

[55] William Mendenhall, Robert J Beaver, and Barbara M Beaver. *Introduction to probability and statistics*. Cengage Learning, 2012.

[56] William Mendenhall, Robert J Beaver, and Barbara M Beaver. *Introduction to probability and statistics*. Cengage Learning, 2012.

[57] Mehryar Mohri, Gary Sivek, and Ananda Theertha Suresh. Agnostic federated learning. In *ICML*, 2019.

[58] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *SOSP*, 2019.

[59] Xingchao Peng, Zijun Huang, Yizhe Zhu, and Kate Saenko. Federated adversarial domain adaptation. In *ICLR*, 2020.

[60] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. A generic communication scheduler for distributed dnn training acceleration. In *SOSP*, 2019.

[61] Qifan Pu, Ganesh Ananthanarayanan, Peter Bodik, Srikanth Kandula, Aditya Akella, Victor Bahl, and Ion Stoica. Low latency Geo-distributed data analytics. In *SIGCOMM*, 2015.

[62] Swaroop Ramaswamy, Om Thakkar, Rajiv Mathews, Galen Andrew, H. Brendan McMahan, and Françoise Beaufays. Training production language models without memorizing user data. In *arxiv.org/abs/2009.10031*, 2020.

[63] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečnỳ, Sanjiv Kumar, and H Brendan McMahan. Adaptive federated optimization. In *ICLR*, 2021.

[64] Edo Roth, Daniel Noble, Brett Hemenway Falk, and Andreas Haeberlen. Honeycrisp: Large-scale differentially private aggregation without a trusted core. In *SOSP*, 2019.

[65] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*, 2018.

[66] Supreeth Shastri, Vinay Banakar, Melissa Wasserman, Arun Kumar, and Vijay Chidambaram. Understanding and benchmarking the impact of GDPR on database systems. In *VLDB*, 2020.

[67] Supreeth Shastri, Melissa Wasserman, and Vijay Chidambaram. The seven sins of personal-data processing systems under GDPR. In *HotCloud*, 2019.

[68] Ananda Theertha Suresh, Felix X. Yu, Sanjiv Kumar, and H. Brendan McMahan. Distributed mean estimation with limited communication. In *ICML*, 2017.

[69] Muhammed Uluyol, Anthony Huang, Ayush Goel, Mosharaf Chowdhury, and Harsha V. Madhyastha. Near-optimal latency versus cost tradeoffs in geo-distributed storage. In *NSDI*, 2020.

[70] Raajay Viswanathan, Ganesh Ananthanarayanan, and Aditya Akella. Clarinet: WAN-aware optimization for analytics queries. In *OSDI*, 2016.

[71] Ashish Vulimiri, Carlo Curino, B Godfrey, J Padhye, and G Varghese. Global analytics in the face of bandwidth and regulatory constraints. In *NSDI*, 2015.

[72] Jianyu Wang and Gauri Joshi. Adaptive communication strategies to achieve the best error-runtime trade-off in local-update SGD. In *MLSys*, 2019.

[73] Kangkang Wang, Rajiv Mathews, Chloe Kiddon, Hubert Eichner, Francoise Beaufays, and Daniel Ramage. Federated evaluation of on-device personalization. In *arxiv.org/abs/1910.10252*, 2019.

[74] Pete Warden. Speech commands: A dataset for limited-vocabulary speech recognition. In *arxiv.org/abs/1804.03209*, 2018.

[75] Mengwei Xu, Jiawei Liu, Yuanqiang Liu, Felix Xiaozhu Lin, Yunxin Liu, and Xuanzhe Liu. A first look at deep learning apps on smartphones. In *WWW*, 2019.

[76] Francis Y Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *NSDI*, 2020.

[77] Timothy Yang, Galen Andrew, Hubert Eichner, Haicheng Sun, Wei Li, Nicholas Kong, Daniel Ramage, and Françoise Beaufays. Applied federated learning: Improving Google keyboard query suggestions. In *arxiv.org/abs/1812.02903*, 2018.

[78] Felix X. Yu, Ankit Singh Rawat, Aditya Krishna Menon, and Sanjiv Kumar. Federated learning with only positive labels. In *ICML*, 2020.

[79] Ben Zhang, Xin Jin, Sylvia Ratnasamy, John Wawrzynek, and Edward A. Lee. AWStream: Adaptive wide-area streaming analytics. In *SIGCOMM*, 2018.

[80] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *CVPR*, 2018.

[81] Peilin Zhao and Tong Zhang. Stochastic optimization with importance sampling for regularized loss minimization. In *ICML*, 2015.